

StatefulSet e Storage Persistente

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare perché le applicazioni stateful richiedono un'identità stabile e storage dedicato per Pod
- Distinguere **StatefulSet** da **Deployment** in termini di naming dei Pod, ordine di scaling e gestione dello storage
- Configurare **volumeClaimTemplates** per ottenere un PVC dedicato e stabile per ogni replica del StatefulSet
- Deployare un database MySQL/PostgreSQL con StatefulSet su OpenShift seguendo le best practice
- Applicare anti-affinity rules per distribuire le repliche su nodi diversi
- Descrivere le implicazioni su ARO: Azure Disk per RWO, Azure Files per RWX, snapshot

Teoria e Concetti Chiave

Perché le Applicazioni Stateful Sono Diverse

Le applicazioni stateful (database, message broker, sistemi di caching distribuiti) hanno requisiti che un **Deployment** standard non soddisfa:

Requisito	Deployment	StatefulSet
Identità Pod stabile	Pod name random (webapp-7d8f9b-xkz2m)	Pod name ordinale stabile (mysql-0 , mysql-1)
Storage per Pod	PVC condiviso tra tutte le repliche	PVC dedicato per ogni replica
Ordine di avvio	Tutte le repliche avviate in parallelo	Ordine sequenziale: mysql-0 prima di mysql-1
Ordine di scale-down	Tutte le repliche rimosse in parallelo	Ordine inverso: mysql-2 rimosso prima di mysql-1
DNS stabile	IP e DNS cambiano ad ogni riavvio	FQDN stabile: mysql-0.mysql-headless.ns.svc.cluster.local

 **Suggerimento:** Un database relazionale come MySQL in modalità primary-replica **non può** essere deployato con un Deployment: le repliche devono sapere qual è il primary (**mysql-0**), avere dati separati e avviarsi in ordine. Questo è esattamente ciò che il StatefulSet fornisce.

volumeClaimTemplates

Invece di referenziare un PVC pre-esistente, un StatefulSet usa `volumeClaimTemplates` per **creare automaticamente un PVC dedicato per ogni Pod**:

```
StatefulSet mysql, replicas: 3
→ Pod mysql-0 → PVC data-mysql-0 → Azure Disk 1
→ Pod mysql-1 → PVC data-mysql-1 → Azure Disk 2
→ Pod mysql-2 → PVC data-mysql-2 → Azure Disk 3
```

Naming dei PVC: `<templateName>-<statefulsetName>-<ordinalIndex>`

Comportamento al delete del StatefulSet:

- I Pod vengono cancellati
- I PVC **rimangono** (non vengono cancellati automaticamente)
- I dati sono preservati; se il StatefulSet viene ricreato, i Pod vengono riassociati agli stessi PVC

 **Attenzione:** La persistenza dei PVC dopo il delete del StatefulSet è una caratteristica di sicurezza. Per pulire completamente i dati è necessario cancellare manualmente i PVC con `oc delete pvc`.

Headless Service per StatefulSet

Un StatefulSet richiede un **Headless Service** (`clusterIP: None`) per fornire DNS stabile per ogni Pod:

```
Service: mysql-headless (clusterIP: None)
→ mysql-0.mysql-headless.database-ns.svc.cluster.local → IP Pod mysql-0
→ mysql-1.mysql-headless.database-ns.svc.cluster.local → IP Pod mysql-1
→ mysql-2.mysql-headless.database-ns.svc.cluster.local → IP Pod mysql-2
```

Il nome DNS di ogni Pod è:

```
<podName>.<serviceName>.<namespace>.svc.cluster.local
```

Ordine di Scaling

Scale-up (da 1 a 3 repliche):

```
mysql-0 creato → Running
mysql-1 creato → Running
mysql-2 creato → Running
```

Scale-down (da 3 a 1 replica):

```
mysql-2 cancellato → Terminated  
mysql-1 cancellato → Terminated  
mysql-0 rimasto → Running
```

Questo garantisce che il Pod `mysql-0` (tipicamente il primary) sia sempre l'ultimo a essere rimosso.

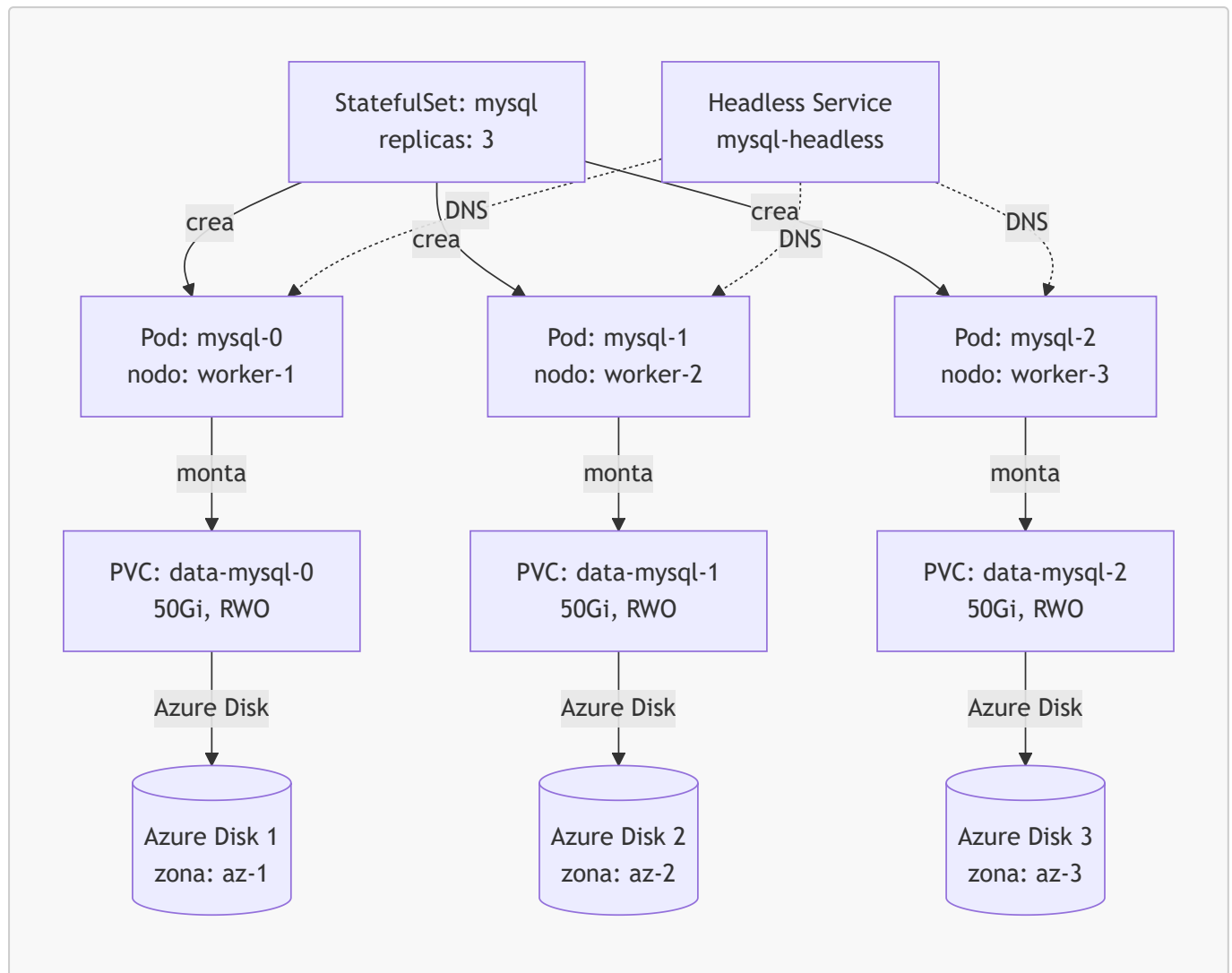
Anti-Affinity per Alta Disponibilità

Per distribuire le repliche su nodi fisici diversi (evitare single point of failure):

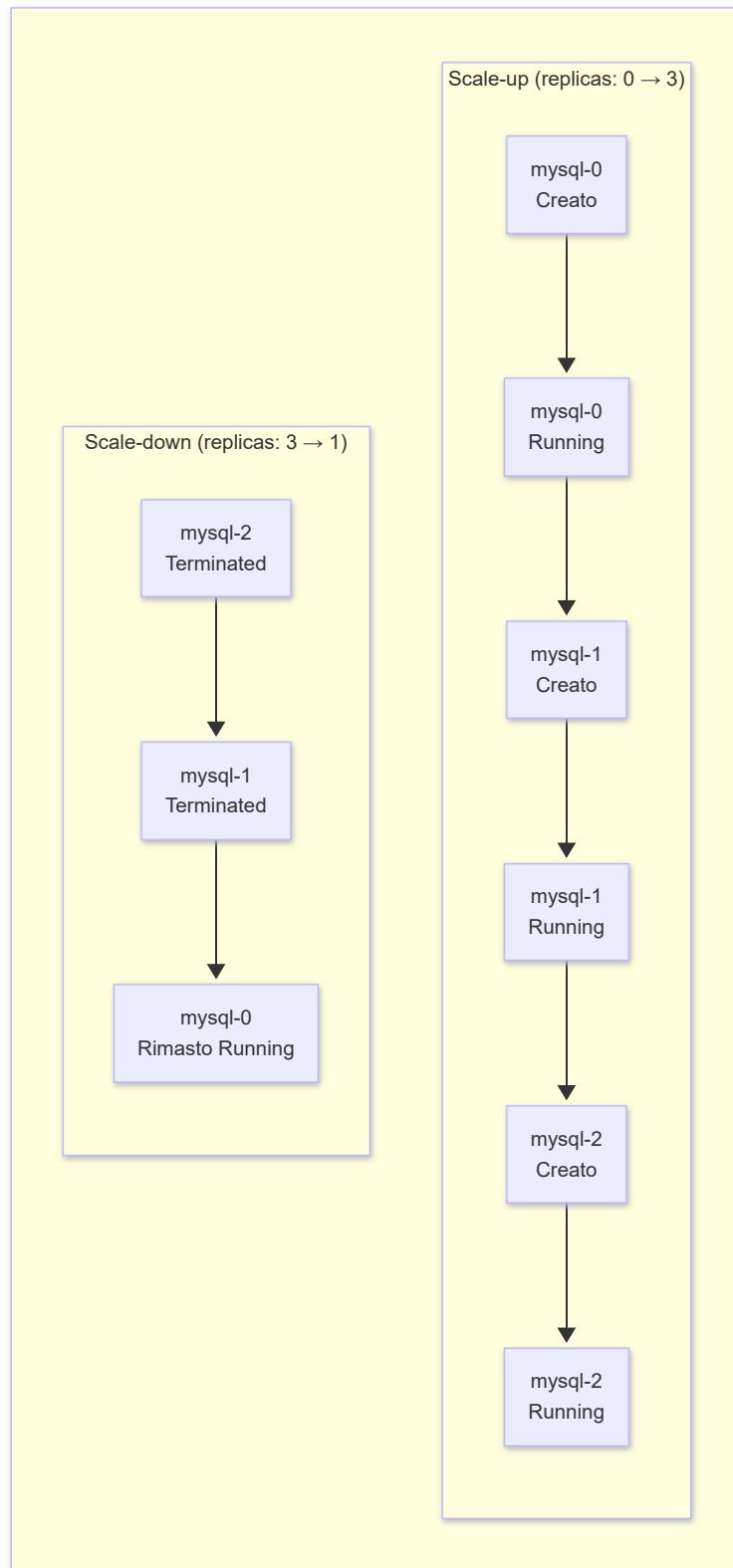
```
affinity:  
  podAntiAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution: # ← HARD: non schedulare  
sullo stesso nodo  
    - labelSelector:  
      matchLabels:  
        app: mysql  
      topologyKey: kubernetes.io/hostname # ← un Pod per nodo
```

Diagrammi

Topologia StatefulSet a 3 Repliche



Ordine Creazione e Cancellazione Pod



Comandi Pratici con Output Atteso

```
# Visualizzare uno StatefulSet e le sue repliche
oc get statefulset mysql -n database-ns
```

```
# Output atteso:
# NAME      READY   AGE
# mysql    3/3     2d
```

```
# Visualizzare i Pod dello StatefulSet con il loro nodo
oc get pods -n database-ns -l app=mysql -o wide
```

```
# Output atteso:
# NAME      READY   STATUS    RESTARTS   AGE   NODE
# mysql-0   1/1     Running   0           2d    worker-1.cluster.example.com
# mysql-1   1/1     Running   0           2d    worker-2.cluster.example.com
# mysql-2   1/1     Running   0           2d    worker-3.cluster.example.com
```

```
# Visualizzare i PVC creati dal volumeClaimTemplates
oc get pvc -n database-ns -l app=mysql
```

```
# Output atteso:
# NAME      STATUS    VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS          AGE
# data-mysql-0   Bound     pvc-1a2b3c4d-...           50Gi
RWX            managed-premium       2d
# data-mysql-1   Bound     pvc-5e6f7a8b-...           50Gi
RWX            managed-premium       2d
# data-mysql-2   Bound     pvc-9c0d1e2f-...           50Gi
RWX            managed-premium       2d
```

```
# Verificare il DNS stabile di ogni Pod tramite il Headless Service
oc exec -it mysql-0 -n database-ns -- nslookup mysql-1.mysql-headless.database-ns.svc.cluster.local
```

```
# Output atteso:
# Name:      mysql-1.mysql-headless.database-ns.svc.cluster.local
# Address: 10.128.5.23
```

```
# Scalare lo StatefulSet a 5 repliche
oc scale statefulset mysql --replicas=5 -n database-ns
```

```
# Output atteso:
# statefulset.apps/mysql scaled

# Osservare la creazione ordinata dei Pod
oc get pods -n database-ns -l app=mysql -w
# NAME          READY   STATUS    RESTARTS   AGE
# mysql-0       1/1     Running   0           2d
# mysql-1       1/1     Running   0           2d
# mysql-2       1/1     Running   0           2d
# mysql-3       0/1     Pending   0           2s  ← mysql-3 creato dopo mysql-2 è
Running
# mysql-3       1/1     Running   0           10s
# mysql-4       0/1     Pending   0           1s  ← mysql-4 creato dopo mysql-3 è
Running
```

```
# Cancellare lo StatefulSet (i PVC rimangono)
oc delete statefulset mysql -n database-ns
```

```
# Output atteso:
# statefulset.apps "mysql" deleted
```

```
# Verificare che i PVC esistano ancora
```

```
oc get pvc -n database-ns
```

```
# NAME          STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
# data-mysql-0   Bound    ...      50Gi       RWO             managed-premium 2d
← ancora presente
# data-mysql-1   Bound    ...      50Gi       RWO             managed-premium 2d
# data-mysql-2   Bound    ...      50Gi       RWO             managed-premium 2d
```

```
# Cancellare manualmente i PVC dopo aver confermato che i dati non servono
oc delete pvc data-mysql-0 data-mysql-1 data-mysql-2 -n database-ns
```

```
# Output atteso:
# persistentvolumeclaim "data-mysql-0" deleted
# persistentvolumeclaim "data-mysql-1" deleted
# persistentvolumeclaim "data-mysql-2" deleted
```

```
# Aggiornare l'immagine di un container in uno StatefulSet (rolling update)
oc set image statefulset/mysql mysql=registry.access.redhat.com/ubi9/ubi-
minimal:9.5 -n database-ns
```

```
# Output atteso:
# statefulset.apps/mysql image updated
# ← l'aggiornamento avviene in ordine inverso: mysql-2, mysql-1, mysql-0
```

Esempi YAML Completi

StatefulSet MySQL con Headless Service e Anti-Affinity

```
# Headless Service: necessario per DNS stabile dei Pod dello StatefulSet
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless          # ← nome referenziato in spec.serviceName
  namespace: database-ns
  labels:
    app: mysql
spec:
  clusterIP: None                # ← Headless: nessun VIP, solo DNS per Pod
  individuali
  ports:
    - name: mysql
      port: 3306
      protocol: TCP
      targetPort: 3306
  selector:
    app: mysql                  # ← seleziona i Pod dello StatefulSet
---
# Service ClusterIP per accesso dall'applicazione (non Headless)
apiVersion: v1
kind: Service
metadata:
  name: mysql                    # ← usato dall'applicazione per connettersi
  namespace: database-ns
  labels:
    app: mysql
spec:
  ports:
    - name: mysql
      port: 3306
      protocol: TCP
      targetPort: 3306
  selector:
    app: mysql
    role: primary                # ← seleziona solo il Pod primary (con label
    role=primary)
---
# StatefulSet MySQL a 3 repliche con storage dedicato e anti-affinity
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
  namespace: database-ns
  labels:
    app: mysql
```



```

spec:
  serviceName: mysql-headless          # ← riferimento al Headless Service
  replicas: 3                          # ← 3 repliche, ognuna con il suo PVC
dedicato
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      # Anti-affinity: garantisce che ogni Pod mysql sia su un nodo diverso
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:  # ← HARD rule:
fallisce se non rispettata
            - labelSelector:
                matchLabels:
                  app: mysql
                topologyKey: kubernetes.io/hostname          # ← un Pod per nodo
      containers:
        - name: mysql
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          command: ["/bin/sh", "-c", "sleep 3600"]
          ports:
            - containerPort: 3306
              name: mysql
          resources:
            requests:
              memory: "512Mi"          # ← minimo garantito per MySQL
              cpu: "250m"
            limits:
              memory: "2Gi"
              cpu: "2000m"
          volumeMounts:
            - name: data                # ← corrisponde al nome nel
volumeClaimTemplates
              mountPath: /var/lib/mysql # ← directory dati MySQL
            - name: config
              mountPath: /etc/mysql/conf.d # ← configurazione MySQL da ConfigMap
              readOnly: true
      # Readiness probe: il Pod non riceve traffico finché MySQL non risponde
      readinessProbe:
        exec:
          command: ["/bin/sh", "-c", "echo 'SELECT 1' | mysql -h 127.0.0.1 -u
root -p${MYSQL_ROOT_PASSWORD}"]
          initialDelaySeconds: 30
          periodSeconds: 10
          failureThreshold: 3
      # Liveness probe: riavvia il container se MySQL non risponde
      livenessProbe:
        exec:
          command: ["/bin/sh", "-c", "mysqladmin ping -h 127.0.0.1 -u root -

```

```
p${MYSQL_ROOT_PASSWORD}"]
  initialDelaySeconds: 30
  periodSeconds: 20
  failureThreshold: 3
env:
- name: MYSQL_ROOT_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mysql-secret      # ← password da Secret, non in chiaro
      key: root-password
volumes:
- name: config
  configMap:
    name: mysql-config      # ← ConfigMap con my.cnf
# volumeClaimTemplates: crea un PVC dedicato per ogni Pod
# PVC creati: data-mysql-0, data-mysql-1, data-mysql-2
volumeClaimTemplates:
- metadata:
    name: data              # ← prefisso nome PVC
    labels:
      app: mysql
spec:
  storageClassName: managed-premium # ← Premium SSD per I/O database
  accessModes:
  - ReadWriteOnce          # ← Azure Disk: un nodo per volta
  resources:
    requests:
      storage: 50Gi        # ← dimensione iniziale per ogni replica
```

Note Specifiche per Azure ARO

Azure Disk vs Azure Files per StatefulSet su ARO

```
# Verifica: Azure Disk (managed-csi) NON supporta RWX
# Se si prova a creare un PVC RWX con managed-csi, il provisioning fallisce
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-rwx
  namespace: test-ns
spec:
  storageClassName: managed-csi
  accessModes:
    - ReadWriteMany          # ← NON supportato da Azure Disk
  resources:
    requests:
      storage: 10Gi
EOF

# Verificare l'errore
oc describe pvc test-rwx -n test-ns | grep -A 5 "Events:"
# Events:
#   Warning  ProvisioningFailed  ... disk.csi.azure.com
#   azuredisk does not support ReadWriteMany access mode
```

```
# Per RWX su ARO, usare azurefile-csi
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: shared-data
  namespace: myproject
spec:
  storageClassName: azurefile-csi    # ← Azure Files supporta RWX
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 100Gi
EOF
```

VolumeSnapshot per StatefulSet su ARO

```
# Verificare le VolumeSnapshotClass disponibili su ARO
oc get volumesnapshotclass
```

```
# Output atteso:
```

# NAME	DRIVER	DELETIONPOLICY	AGE
# csi-azuredisk-vsc	disk.csi.azure.com	Delete	5d
# csi-azurefile-vsc	file.csi.azure.com	Delete	5d

```
# Creare uno snapshot del PVC del Pod primario (mysql-0)
```

```
oc apply -f - <<EOF
```

```
apiVersion: snapshot.storage.k8s.io/v1
```

```
kind: VolumeSnapshot
```

```
metadata:
```

```
  name: mysql-0-snapshot-$(date +%Y%m%d)
```

```
  namespace: database-ns
```

```
spec:
```

```
  volumeSnapshotClassName: csi-azuredisk-vsc
```

```
  source:
```

```
    persistentVolumeClaimName: data-mysql-0
```

```
EOF
```

⚠ Attenzione: Per applicazioni stateful come database, è fortemente consigliato eseguire uno snapshot coordinato con la quiescence dell'applicazione (es. FLUSH TABLES WITH READ LOCK per MySQL) prima di creare il VolumeSnapshot, per garantire la consistenza dei dati.

Riepilogo dei Punti Chiave

-

Visualizzare uno StatefulSet e le sue repliche

```
oc get statefulset mysql -n database-ns
```

Output atteso:

NAME READY AGE

mysql 3/3 2d

```
```bash
Visualizzare i Pod dello StatefulSet con il loro nodo
oc get pods -n database-ns -l app=mysql -o wide

Output atteso:
NAME READY STATUS RESTARTS AGE NODE
mysql-0 1/1 Running 0 2d worker-1.cluster.example.com
mysql-1 1/1 Running 0 2d worker-2.cluster.example.com
mysql-2 1/1 Running 0 2d worker-3.cluster.example.com
```

```
Visualizzare i PVC creati dal volumeClaimTemplates
oc get pvc -n database-ns -l app=mysql

Output atteso:
NAME STATUS VOLUME CAPACITY
ACCESS MODES STORAGECLASS AGE
data-mysql-0 Bound pvc-1a2b3c4d-... 50Gi
RWX managed-premium 2d
data-mysql-1 Bound pvc-5e6f7a8b-... 50Gi
RWX managed-premium 2d
data-mysql-2 Bound pvc-9c0d1e2f-... 50Gi
RWX managed-premium 2d
```

```
Verificare il DNS stabile di ogni Pod tramite il Headless Service
oc exec -it mysql-0 -n database-ns -- nslookup mysql-1.mysql-headless.database-
```

```
ns.svc.cluster.local
```

```
Output atteso:
```

```
Name: mysql-1.mysql-headless.database-ns.svc.cluster.local
```

```
Address: 10.128.5.23
```

```
Scalare lo StatefulSet a 5 repliche
```

```
oc scale statefulset mysql --replicas=5 -n database-ns
```

```
Output atteso:
```

```
statefulset.apps/mysql scaled
```

```
Osservare la creazione ordinata dei Pod
```

```
oc get pods -n database-ns -l app=mysql -w
```

```
NAME READY STATUS RESTARTS AGE
mysql-0 1/1 Running 0 2d
mysql-1 1/1 Running 0 2d
mysql-2 1/1 Running 0 2d
mysql-3 0/1 Pending 0 2s ← mysql-3 creato dopo mysql-2 è
Running
mysql-3 1/1 Running 0 10s
mysql-4 0/1 Pending 0 1s ← mysql-4 creato dopo mysql-3 è
Running
```

```
Cancellare lo StatefulSet (i PVC rimangono)
```

```
oc delete statefulset mysql -n database-ns
```

```
Output atteso:
```

```
statefulset.apps "mysql" deleted
```

```
Verificare che i PVC esistano ancora
```

```
oc get pvc -n database-ns
```

```
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS AGE
data-mysql-0 Bound ... 50Gi RWO managed-premium 2d
← ancora presente
data-mysql-1 Bound ... 50Gi RWO managed-premium 2d
data-mysql-2 Bound ... 50Gi RWO managed-premium 2d
```

```
Cancellare manualmente i PVC dopo aver confermato che i dati non servono
```

```
oc delete pvc data-mysql-0 data-mysql-1 data-mysql-2 -n database-ns
```

```
Output atteso:
```

```
persistentvolumeclaim "data-mysql-0" deleted
```

```
persistentvolumeclaim "data-mysql-1" deleted
```

```
persistentvolumeclaim "data-mysql-2" deleted
```

```
Aggiornare l'immagine di un container in uno StatefulSet (rolling update)
oc set image statefulset/mysql mysql=registry.access.redhat.com/ubi9/ubi-
minimal:9.5 -n database-ns
```

```
Output atteso:
```

```
statefulset.apps/mysql image updated
```

```
← l'aggiornamento avviene in ordine inverso: mysql-2, mysql-1, mysql-0
```

---

## Esempi YAML Completi

### StatefulSet MySQL con Headless Service e Anti-Affinity

```
Headless Service: necessario per DNS stabile dei Pod dello StatefulSet
apiVersion: v1
kind: Service
metadata:
 name: mysql-headless # ← nome referenziato in spec.serviceName
 namespace: database-ns
 labels:
 app: mysql
spec:
 clusterIP: None # ← Headless: nessun VIP, solo DNS per Pod
 individuali
 ports:
 - name: mysql
 port: 3306
 protocol: TCP
 targetPort: 3306
 selector:
 app: mysql # ← seleziona i Pod dello StatefulSet

Service ClusterIP per accesso dall'applicazione (non Headless)
apiVersion: v1
kind: Service
metadata:
 name: mysql # ← usato dall'applicazione per connettersi
 namespace: database-ns
 labels:
 app: mysql
spec:
 ports:
 - name: mysql
 port: 3306
 protocol: TCP
 targetPort: 3306
 selector:
 app: mysql
 role: primary # ← seleziona solo il Pod primary (con label
 role=primary)

StatefulSet MySQL a 3 repliche con storage dedicato e anti-affinity
apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: mysql
 namespace: database-ns
 labels:
 app: mysql
```



```

spec:
 serviceName: mysql-headless # ← riferimento al Headless Service
 replicas: 3 # ← 3 repliche, ognuna con il suo PVC
dedicato
 selector:
 matchLabels:
 app: mysql
 template:
 metadata:
 labels:
 app: mysql
 spec:
 # Anti-affinity: garantisce che ogni Pod mysql sia su un nodo diverso
 affinity:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution: # ← HARD rule:
fallisce se non rispettata
 - labelSelector:
 matchLabels:
 app: mysql
 topologyKey: kubernetes.io/hostname # ← un Pod per nodo
 containers:
 - name: mysql
 image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
 command: ["/bin/sh", "-c", "sleep 3600"]
 ports:
 - containerPort: 3306
 name: mysql
 resources:
 requests:
 memory: "512Mi" # ← minimo garantito per MySQL
 cpu: "250m"
 limits:
 memory: "2Gi"
 cpu: "2000m"
 volumeMounts:
 - name: data # ← corrisponde al nome nel
volumeClaimTemplates
 mountPath: /var/lib/mysql # ← directory dati MySQL
 - name: config
 mountPath: /etc/mysql/conf.d # ← configurazione MySQL da ConfigMap
 readOnly: true
 # Readiness probe: il Pod non riceve traffico finché MySQL non risponde
 readinessProbe:
 exec:
 command: ["/bin/sh", "-c", "echo 'SELECT 1' | mysql -h 127.0.0.1 -u
root -p${MYSQL_ROOT_PASSWORD}"]
 initialDelaySeconds: 30
 periodSeconds: 10
 failureThreshold: 3
 # Liveness probe: riavvia il container se MySQL non risponde
 livenessProbe:
 exec:
 command: ["/bin/sh", "-c", "mysqladmin ping -h 127.0.0.1 -u root -

```

```
p${MYSQL_ROOT_PASSWORD}"]
 initialDelaySeconds: 30
 periodSeconds: 20
 failureThreshold: 3
env:
- name: MYSQL_ROOT_PASSWORD
 valueFrom:
 secretKeyRef:
 name: mysql-secret # ← password da Secret, non in chiaro
 key: root-password
volumes:
- name: config
 configMap:
 name: mysql-config # ← ConfigMap con my.cnf
volumeClaimTemplates: crea un PVC dedicato per ogni Pod
PVC creati: data-mysql-0, data-mysql-1, data-mysql-2
volumeClaimTemplates:
- metadata:
 name: data # ← prefisso nome PVC
 labels:
 app: mysql
spec:
 storageClassName: managed-premium # ← Premium SSD per I/O database
 accessModes:
 - ReadWriteOnce # ← Azure Disk: un nodo per volta
 resources:
 requests:
 storage: 50Gi # ← dimensione iniziale per ogni replica
```

---

# Note Specifiche per Azure ARO

## Azure Disk vs Azure Files per StatefulSet su ARO

```
Verifica: Azure Disk (managed-csi) NON supporta RWX
Se si prova a creare un PVC RWX con managed-csi, il provisioning fallisce
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: test-rwx
 namespace: test-ns
spec:
 storageClassName: managed-csi
 accessModes:
 - ReadWriteMany # ← NON supportato da Azure Disk
 resources:
 requests:
 storage: 10Gi
EOF

Verificare l'errore
oc describe pvc test-rwx -n test-ns | grep -A 5 "Events:"
Events:
Warning ProvisioningFailed ... disk.csi.azure.com
azuredisk does not support ReadWriteMany access mode
```

```
Per RWX su ARO, usare azurefile-csi
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: shared-data
 namespace: myproject
spec:
 storageClassName: azurefile-csi # ← Azure Files supporta RWX
 accessModes:
 - ReadWriteMany
 resources:
 requests:
 storage: 100Gi
EOF
```

## VolumeSnapshot per StatefulSet su ARO

```
Verificare le VolumeSnapshotClass disponibili su ARO
oc get volumesnapshotclass
```

```
Output atteso:
```

# NAME	DRIVER	DELETIONPOLICY	AGE
# csi-azuredisk-vsc	disk.csi.azure.com	Delete	5d
# csi-azurefile-vsc	file.csi.azure.com	Delete	5d

```
Creare uno snapshot del PVC del Pod primario (mysql-0)
```

```
oc apply -f - <<EOF
```

```
apiVersion: snapshot.storage.k8s.io/v1
```

```
kind: VolumeSnapshot
```

```
metadata:
```

```
 name: mysql-0-snapshot-$(date +%Y%m%d)
```

```
 namespace: database-ns
```

```
spec:
```

```
 volumeSnapshotClassName: csi-azuredisk-vsc
```

```
 source:
```

```
 persistentVolumeClaimName: data-mysql-0
```

```
EOF
```

**⚠ Attenzione:** Per applicazioni stateful come database, è fortemente consigliato eseguire uno snapshot coordinato con la quiescence dell'applicazione (es. FLUSH TABLES WITH READ LOCK per MySQL) prima di creare il VolumeSnapshot, per garantire la consistenza dei dati.

## Riepilogo dei Punti Chiave

- **StatefulSet** fornisce identità stabile per ogni Pod (nome ordinale fisso), storage dedicato via **volumeClaimTemplates** e ordine di scaling garantito: indispensabile per database e applicazioni stateful
  - **volumeClaimTemplates** crea automaticamente un PVC per ogni replica con nome `<template>-<statefulset>-<ordinal>`; i PVC sopravvivono alla cancellazione del StatefulSet
  - Il **Headless Service** (`clusterIP: None`) fornisce DNS stabile per ogni Pod: `<pod>.<service>.<ns>.svc.cluster.local`
  - Su ARO, usare **managed-premium** (Azure Disk Premium SSD) per database ad alto I/O; **azurefile-csi** per storage condiviso RWX
  - L'anti-affinity (**requiredDuringSchedulingIgnoredDuringExecution**) garantisce che ogni replica sia su un nodo fisico diverso, aumentando la disponibilità
  - Dopo il delete del StatefulSet, cancellare i PVC manualmente solo dopo aver verificato che i dati non siano più necessari
-

## Domande di Verifica

**1. Uno StatefulSet `postgres` con 3 repliche usa `volumeClaimTemplates` con name `pgdata`. Qual è il nome del PVC della terza replica?**

- A) `pgdata-postgres-2` ✓
- B) `pgdata-postgres-3`
- C) `postgres-pgdata-2`
- D) `pgdata-2`

**2. Un DBA cancella accidentalmente il StatefulSet `mysql` in produzione. Cosa succede ai dati?**

- A) I dati vengono cancellati insieme ai Pod
- B) I dati sono al sicuro nei PVC che rimangono anche dopo il delete del StatefulSet ✓
- C) I dati vengono spostati automaticamente in un PVC di backup
- D) I dati vengono cancellati solo se la `reclaimPolicy` è `Delete`

**3. Quale oggetto Kubernetes è obbligatorio per un StatefulSet per fornire DNS stabile per i singoli Pod?**

- A) Service di tipo LoadBalancer
- B) Service di tipo ClusterIP normale
- C) Headless Service (clusterIP: None) ✓
- D) Ingress con wildcard

**4. Su ARO, si vuole deployare un sistema di file storage condiviso tra tutte le repliche di un StatefulSet (accesso RWX). Quale StorageClass usare?**

- A) `managed-csi`
- B) `managed-premium`
- C) `azurefile-csi` ✓
- D) `local-storage`

**5. Un'operazione di scale-down di uno StatefulSet MySQL da 5 a 2 repliche procede in quale ordine?**

- A) `mysql-0`, `mysql-1`, `mysql-2` vengono cancellati contemporaneamente
- B) `mysql-4`, `mysql-3`, `mysql-2` vengono cancellati in ordine decrescente ✓
- C) `mysql-2`, `mysql-3`, `mysql-4` vengono cancellati in ordine crescente
- D) L'ordine è casuale, dipende dalla disponibilità dei nodi